Addis Ababa University

College of Technology and Built Environment
School of Information Technology and Engineering

Program: Bachelor of Software Engineering

Course Title: Fundamental of Data structure and analysis

No	Section	Contents
1	Module:	
	1.1. Module Title	Fundamental of Data structure and analysis
	1.2. Type of Module	Core
2	Core competencies	**Understanding Basic Data Structures:** • Arrays, Linked Lists, Stacks, Queues, Trees, Graphs, Hash Tables. • Ability to implement these data structures in a programming language. **Algorithm Analysis:** • Time and Space Complexity (Big O, Big Θ, Big Ω Notation). • Basic sorting and searching algorithms (Bubble Sort, Merge Sort, Quick Sort, Binary Search). • Understanding of Divide and Conquer, Greedy Algorithms, and Dynamic Programming. **Advanced Data Structures:** • Trees: Binary Search Trees, AVL Trees, B-Trees. • Graph Algorithms: DFS, BFS, Dijkstra's, Kruskal's, and Prim's algorithms. • Hashing Techniques and Collision Resolution. **Practical Problem Solving:** • Designing efficient algorithms for specific problems. • Using data structures to optimize algorithms.

		• Case studies of real-world problems and their solutions. **Programming Skills:** • Proficiency in a high-level programming language (e.g., Python, Java, C++). • Ability to translate algorithms into code. • Debugging and testing data structure implementations. **Critical Thinking and Analysis:** • Comparing different data structures and algorithms for efficiency. • Analyzing trade-offs in choosing data structures for specific applications.
3	Course Type (Lab, e-learning, Seminar, Practicum/ Field Attachment, Senior Essay/Thesis/Dissertation)	**Lecture** (Core concepts and theory) **Lab** (Practical implementation and coding exercises) **Seminar** (Presentations on advanced topics and case studies) **Practicum/Field** (Optional: Projects applying data structures to real-world problems)
4	Course Identifiers	
	4.1. Course Code	SITE-2082
	4.2. Credit Hrs/ECTS/	5
5	Semester and Year	Semester II
6	Prerequisite	Introduction to Programming
7	Instructor • Name • Contact information • Office and Office Hrs	
8	Course Description	**Fundamentals of Data Structures and Algorithm Analysis** is a core course designed to introduce undergraduate Software Engineering students to the fundamental concepts of data structures and

		algorithms. This course emphasizes the importance of efficient data organization and algorithmic problem-solving techniques, which are essential for developing robust and scalable software applications. Students will learn to analyze the time and space complexity of algorithms, design and implement various data structures, and apply algorithmic techniques to solve real-world problems. Through a combination of lectures, labs, and hands-on projects, students will gain practical experience in coding and problem-solving, preparing them for more advanced courses and industry challenges.
9	Integrity Statement	The Department of Software Engineering at AAiT is committed to upholding the highest standards of academic integrity. This integrity statement outlines the expectations and responsibilities for both students and instructors in the "Fundamentals of Data Structures and Algorithm Analysis" course.
10	Students with Disability	The Department of Software Engineering at AAiT is committed to providing an inclusive and accessible learning environment for all students, including those with disabilities. We strive to ensure that all students have equal opportunities to participate in and benefit from our courses.
11	Learning Outcomes	By the end of this course, students will be able to: • Implement Fundamental Data Structures: Design, implement, and manipulate basic data structures such as arrays, linked lists, stacks, queues, trees, and graphs in a high-level programming language. • Analyze Algorithm Efficiency: Evaluate the time and space complexity of algorithms using Big O, Big Θ, and Big Ω notations, and understand the trade-offs between different algorithms and data structures. • Apply Sorting and Searching Algorithms: Implement and analyze common sorting (e.g., Quick Sort, Merge Sort) and searching algorithms (e.g., Binary Search), and apply them to solve computational problems efficiently.

		• Solve Complex Problems Using Advanced Data Structures: Utilize advanced data structures such as AVL trees, B-trees, and hash tables to address more complex computational challenges. • Design Algorithms for Real-World Applications: Develop and optimize algorithms for practical problems, applying techniques like divide and conquer, dynamic programming, and greedy algorithms. • Work Collaboratively on Projects: Engage in group projects that require the application of data structures and algorithms to real-world scenarios, effectively communicating and presenting solutions. • Demonstrate Critical Thinking: Critically analyze different data structures and algorithms to choose the most appropriate solution for a given problem, considering efficiency and scalability.
12	Course Contents	**Week 1: Introduction to Data Structures and Algorithms** • **Lecture:** Overview of Data Structures and Algorithms • **Lab:** Setting up the development environment **Week 2: Arrays and Linked Lists** • **Lecture:** Arrays, Linked Lists (Singly, Doubly, Circular) • **Lab:** Implementing Arrays and Linked Lists **Week 3: Algorithm Analysis** • **Lecture:** Time and Space Complexity, Big O Notation • **Lab:** Analyzing Algorithm Complexity **Week 4: Stacks and Queues** • **Lecture:** Stacks, Queues, Deques • **Lab:** Implementing Stacks and Queues

		Week 5: Trees • **Lecture:** Binary Trees, Binary Search Trees, AVL Trees • **Lab:** Implementing Binary Trees and Binary Search Trees **Week 6: Graphs** • **Lecture:** Graph Representation, Graph Traversal (BFS, DFS) • **Lab:** Implementing Graphs and Traversal Algorithms **Week 7: Hash Tables** • **Lecture:** Hash Functions, Collision Resolution • **Lab:** Implementing Hash Tables **Week 8: Divide and Conquer** • **Lecture:** Merge Sort, Quick Sort, Binary Search • **Lab:** Implementing Divide and Conquer Algorithms **Week 9: Dynamic Programming** • **Lecture:** Fibonacci Sequence, Knapsack Problem, Longest Common Subsequence • **Lab:** Implementing Dynamic Programming Algorithms **Week 10: Greedy Algorithms** • **Lecture:** Activity Selection, Huffman Coding, Fractional Knapsack • **Lab:** Implementing Greedy Algorithms **Week 11: Backtracking** • **Lecture:** N-Queens Problem, Sudoku Solver • **Lab:** Implementing Backtracking Algorithms **Week 12: Advanced Topics in Data Structures** • **Lecture:** Tries, B-Trees, Red-Black Trees

		• **Lab:** Implementing Advanced Data Structures **Week 13: Advanced Topics in Algorithms** • **Lecture:** Network Flow, Minimum Spanning Trees, Shortest Path Algorithms • **Lab:** Implementing Advanced Algorithms **Week 14: Review and Practice** • **Lecture:** Review of Key Concepts • **Lab:** Practice Problems and Project Work **Week 15: Final Project and Exam** • **Lecture:** Final Review Session • **Lab:** Final Project Presentations • **Exam:** Comprehensive Final Exam
13	Teaching and learning techniques	**Interactive Lectures**: Engage students with live coding demos and Q&A to explain data structures and algorithms. **Hands-on Lab Sessions**: Provide practical coding exercises to implement and test data structures and algorithms. **Problem-Based Learning (PBL):** Use real-world problems to apply and reinforce concepts in data structures and algorithms. **Continuous Assessment and Feedback**: Regular quizzes and assignments with prompt feedback to track progress. **Discussion Forums and Online Collaboration**: Use online platforms for continuous discussion and collaborative problem-solving. **Flipped Classroom**: Assign pre-class materials, using class time for active problem-solving and discussions.
14	Assessments	**Lab Assignments:** • Practical coding tasks where students implement and test data structures and algorithms.

		Quizzes: • Short, regular quizzes on theoretical concepts and problem-solving techniques. **Midterm Exam**: • A comprehensive exam covering the first half of the course, including data structures and basic algorithms. **Project Work:** • A significant assignment where students design, implement, and optimize solutions to a complex problem using data structures and algorithms. **Final Exam:** • A comprehensive exam covering all course material, including advanced data structures and algorithms.
15	Course Policy	• Attendance is mandatory according to AAU senate legislation. • Late submissions will incur penalties. • Academic honesty is strictly enforced. • Group work is encouraged for projects and collaborative assignments. • Students should arrive on time, be prepared for class, and minimize disruptions.
16	Learning Resources	
	16.1. Textbooks	"Algorithms" by Robert Sedgewick and Kevin Wayne (4th Edition, 2021)
	16.2. References	• Data Structures and Algorithms in C++" by Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser (2nd Edition, 2011) • "Introduction to Algorithms" by Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest,

		and Clifford Stein (4th Edition, 2022) • "Data Structures and Algorithms in Java" by Robert Lafore